

Proposed Solution

CrediShield AI — Credit Card Approval Prediction System

1. Solution Overview

CrediShield AI is an intelligent credit card approval prediction platform that combines machine learning classification with a premium web dashboard. The system ingests applicant profile data, processes it through a trained Gradient Boosting model, and returns an instant approve/reject decision with confidence scoring and risk grading.

Core Value:

Replace 7–14 day manual credit review with **sub-second ML predictions** achieving **98.29% accuracy** and **99.14% F1-score**, delivered through a stunning glassmorphism web interface.

2. Solution Components

Component 1: ML Training Pipeline (`train.py`)

Purpose: Automates the entire machine learning lifecycle from raw data to serialized, production-ready model artifacts.

Pipeline Steps:

Step	Action	Output
1	Load <code>application_record.csv</code> and <code>credit_record.csv</code> from data/	Raw DataFrames
2	Engineer binary target labels from payment status codes	<code>TARGET</code> column (0 = Rejected, 1 = Approved)
3	Merge datasets on <code>ID</code> ; deduplicate applications	Combined DataFrame
4	Clean data: fill missing occupations with 'Unknown', remove income outliers (IQR)	Clean DataFrame
5	Create derived features: <code>AGE</code> , <code>YEARS_EMPLOYED</code> , <code>UNEMPLOYED</code> , <code>INCOME_PER_MEMBER</code> , <code>INCOME_CAT</code> , <code>AGE_CAT</code> , <code>EMPLOYED_CAT</code>	Enriched DataFrame
6	Encode 11 categorical columns using <code>LabelEncoder</code>	Integer-encoded DataFrame
7	Scale 6 numerical columns using <code>StandardScaler</code>	Normalized DataFrame
8	Stratified 80/20 train-test split	<code>X_train</code> , <code>X_test</code> , <code>y_train</code> , <code>y_test</code>
9	Train 6 models: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, Balanced Random Forest	6 fitted models
10	Evaluate all models on Accuracy, Precision, Recall, F1-Score, ROC AUC	Comparison metrics
11	Auto-select winner by highest F1-Score	Gradient Boosting (99.14% F1)
12	Serialize model, scaler, encoders, feature names, metrics, and stats to <code>models/</code>	<code>6.joblib</code> + <code>2.json</code> files

Component 2: Flask REST Backend (`app.py`)

Purpose: Serves as the API layer connecting the frontend dashboard to the ML inference engine.

Endpoints:

Endpoint	Method	Input	Output	Description
<code>/</code>	GET	—	HTML page	Renders the single-page CrediShield AI dashboard
<code>/predict</code>	POST	JSON (14 fields)	<code>{approved, confidence, risk_level}</code>	Preprocesses inputs, runs inference, returns prediction
<code>/metrics</code>	GET	—	<code>{metrics: {...}, stats: {...}}</code>	Returns model comparison data and training statistics for charts

Key Architecture Decisions:

- **Dynamic Feature Alignment:** Reads `feature_names.joblib` to assemble the feature vector in the exact order the model was trained on — prevents order mismatch errors.
- **DataFrame-based Scaling:** Passes a named Pandas DataFrame to `StandardScaler.transform()` instead of raw lists — suppresses scikit-learn's feature name warnings.
- **Auto-Training Fallback:** If model files are missing on startup (lines 15–18), automatically imports and runs `train.py` — self-healing deployment.
- **Graceful Error Handling:** All endpoints wrapped in `try-except` blocks with descriptive JSON error messages.

Component 3: Web Dashboard (`index.html` + `style.css` + `script.js`)

Purpose: Provides a visually stunning, interactive interface for non-technical bank staff to submit applications and view results.

UI Features:

Feature	Technology	Description
Obsidian Glassmorphism Theme	CSS3 <code>backdrop-filter</code>	Deep-space dark panels with <code>blur(25px)</code> frosted glass effect and light refraction borders
Ambient Background Glows	CSS animations	Three floating neon gradient blobs with slow translation animations creating depth
Tabbed Navigation	JavaScript	4-tab layout: Predict, Model Metrics, Visual Analytics, Prediction History
SVG Confidence Gauge	SVG + JS animation	Circular progress ring using <code>stroke-dashoffset</code> transitions with green (approved) / red (rejected) gradients
Numeric Count-Up	JavaScript <code>setInterval</code>	Confidence percentage animates from 0% to final value in sync with the ring
Interactive Charts	Chart.js 4.x (CDN)	4 chart types: feature importance (bar), approval ratio (doughnut), income distribution (bar), age groups (pie)
Prediction History	localStorage	Persistent table with transaction IDs, timestamps, decisions, and per-entry PDF downloads

PDF Report Generator	iframe + window.print()	Branded "CrediShield AI Credit Assessment Letter" with full applicant metadata
Theme Toggle	localStorage + CSS variables	Dark/light switch with sun/moon SVG icons; preference persisted across sessions
Custom Scrollbar	CSS ::-webkit-scrollbar	Themed scrollbar matching the obsidian design

3. Solution Workflow Diagram



4. Models Compared

#	Algorithm	Type	Accuracy	F1 Score	ROC AUC	Selected?
1	Logistic Regression	Linear	91.45%	95.53%	93.11%	✗
2	Decision Tree	Tree-based	95.73%	97.69%	95.56%	✗
3	Random Forest	Ensemble (Bagging)	97.85%	98.86%	96.98%	✗
4	Gradient Boosting	Ensemble (Boosting)	98.29%	99.14%	97.53%	☑ Winner
5	XGBoost	Ensemble (Boosting)	97.91%	98.91%	96.50%	✗
6	Balanced Random Forest	Ensemble (Balanced)	86.12%	86.93%	92.84%	✗

Selection Criterion: Highest F1-Score (balances precision and recall, critical for imbalanced datasets)

5. Deployment Strategy

Aspect	Details
Platform	Render.com (Free Tier Web Service)
Server	Gunicorn 22.0.0 (WSGI HTTP server)
Start Command	<code>gunicorn app:app</code> (defined in Procfile)
Auto-Deploy	GitHub webhook triggers rebuild on every push to main branch
Python Version	3.12 (pinned in .python-version)
Dependencies	Pinned in requirements.txt to prevent version mismatch deserialization errors

Cold Start Free-tier apps sleep after 15 min inactivity; ~30 second wake-up time on first request
URL <https://credishield-ai.onrender.com> (or as assigned)

Project Name: CrediShield AI Author: Dinesh (dineshTech-creator) Date: July 2026
--